

Proyecto de Grado: IA al servicio de las comunidades

Maestría en Analítica para la Inteligencia de Negocios

Nombres: María Fernanda Izquierdo Aparicio & Ana María Ochoa Muñoz

Fecha: 26 de Abril de 2025

Este modelo forma parte del proyecto de grado de la Maestría en Analítica para la Inteligencia de Negocios de la Pontificia Universidad Javeriana, enfocado en la aplicación de Inteligencia Artificial al servicio de las comunidades. Su objetivo es clasificar imágenes de plantas en función de su especie usando una arquitectura pre-entrenada RESNET

1. Importe de Librerías

```
In [63]: import json

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.image as mpimg

import os
import zipfile
import shutil
import seaborn as sns
import datetime
import cv2

import keras
from keras import models
from keras import layers

import random

import tensorflow as tf
from tensorflow.keras.models import Model
from tensorflow.keras import Layer
from tensorflow.keras.layers import Input, Dense
from tensorflow.keras.preprocessing import image
```

```

from tensorflow.keras.preprocessing.image import ImageDataGenerator
from tensorflow.keras.preprocessing.image import img_to_array, load_img
from tensorflow.keras.callbacks import ReduceLROnPlateau
from tensorflow.keras.callbacks import ModelCheckpoint, EarlyStopping, ReduceLROnPlateau
from tensorflow.keras.applications import EfficientNetB0
from tensorflow.keras.optimizers import Adam
from tensorflow.keras.applications import ResNet50
from tensorflow.keras.layers import GlobalAveragePooling2D
from tensorflow.keras.utils import to_categorical

from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, f1_score
from sklearn.metrics import confusion_matrix, accuracy_score, ConfusionMatrixDisplay
from sklearn.metrics import roc_curve, auc
from sklearn.metrics import classification_report

from PIL import Image # Librería para abrir y manipular imágenes

# ignoring warnings
import warnings
warnings.simplefilter("ignore")

```

```

In [64]: #Medir_tiempo_ejecucion():
         inicio = datetime.datetime.now()
         print(f"Inicio: {inicio.strftime('%Y-%m-%d %H:%M:%S.%f')}")

```

Inicio: 2025-05-12 04:23:34.397593

2. Definición de Rutas y Parámetros

```

In [66]: data_dir = 'Datos Finales' # Reemplaza con la ruta real
         metadata_file = 'Metadatos_Finales.csv' # Reemplaza con la ruta real
         image_height = 150
         image_width = 150
         batch_size = 15
         num_classes = None # Se determinará automáticamente del archivo CSV

```

3. Cargar y procesar los metadatos

```

In [68]: try:
         metadata = pd.read_csv(metadata_file, sep=";")
         metadata["Planta_Condición_Enfermedad"] = metadata["Planta_Estado_Español"]
         metadata = metadata.rename(columns={'Nombre del archivo': 'image_id'})

         # Asegúrate de que las columnas 'filename' (o el nombre de tu columna de
         # y 'label' (o el nombre de tu columna de etiquetas) existan.

```

```

if 'image_id' not in metadata.columns or 'Planta_Condición_Enfermedad' not in metadata.columns:
    raise ValueError("El archivo CSV debe contener las columnas 'image_id' y 'Planta_Condición_Enfermedad'")
class_names = metadata['Planta_Condición_Enfermedad'].unique().tolist()
num_classes = len(class_names)
print(f"Se encontraron {num_classes} clases: {class_names}")
except FileNotFoundError:
    print(f"Error: No se encontró el archivo de metadatos en '{metadata_file}'")
    exit()
except ValueError as e:
    print(f"Error al procesar el archivo CSV: {e}")
    exit()

```

Se encontraron 37 clases: ['Yuca_Enferma_Pudrición de la yuca', 'Yuca_Enferma_Manchas amarillas en la yuca', 'Yuca_Enferma_Venas marrones de la yuca', 'Yuca_Enferma_Manchas verdes en la yuca', 'Yuca_Saludable_Saludable', 'Fresa_Saludable_Saludable', 'Papa_Enferma_Quemado de la papa', 'Maíz_Saludable_Saludable', 'Tomate_Enferma_Pinta en ojo de buey', 'Papa_Enferma_Pudrición de la papa', 'Tomate_Enferma_Pudrición de la papa', 'Tomate_Enferma_Moteado del tomate', 'Tomate_Enferma_Hongo blanco del tomate', 'Manzana_Enferma_Hongo del manzano', 'Tomate_Enferma_Puntos negros del tomate', 'Tomate_Enferma_Quemado de la papa', 'Maíz_Enferma_Moho rojo del maíz', 'Tomate_Saludable_Saludable', 'Tomate_Enferma_Hojas enrolladas del tomate', 'Manzana_Enferma_Costra de la manzana', 'Maíz_Enferma_Quemazón del maíz', 'Tomate_Enferma_Polvillo rojo', 'Tomate_Enferma_Puntitos marrones en las hojas', 'Maíz_Enferma_Pinta gris', 'Manzana_Enferma_Pudrición del fruto', 'Manzana_Saludable_Saludable', 'Fresa_Enferma_Hojas quemadas de la fresa', 'Papa_Saludable_Saludable', 'Coliflor_Enferma_Moho azul', 'Pepino Cohombro_Saludable_Saludable', 'Fríjol_Enferma_Mancha angular', 'Coliflor_Saludable_Saludable', 'Coliflor_Enferma_Pudrición del fruto', 'Fríjol_Enferma_Oxido de frijol', 'Coliflor_Enferma_Podredumbre blanda', 'Fríjol_Saludable_Saludable', 'Pepino Cohombro_Enferma_Enferma']

```

In [69]: train, test = train_test_split(metadata, test_size=0.2, random_state=42, shuffle=True)
          train.shape, test.shape

```

```

Out[69]: ((42354, 13), (10589, 13))

```

4. Crear un generador de datos

```

In [71]: from sklearn.utils.class_weight import compute_class_weight
          import numpy as np

          # Obtener las clases del DataFrame
          labels = train['Planta_Condición_Enfermedad']
          classes = np.unique(labels)

          # Calcular pesos balanceados
          class_weights = compute_class_weight(class_weight='balanced', classes=classes)
          class_weights_dict = dict(zip(classes, class_weights))

```

```
# Verificar pesos
print("Pesos por clase:", class_weights_dict)
```

```
Pesos por clase: {'Coliflor_Enferma_Moho azu': 8.061286638751428, 'Coliflor_Enferma_Podredumbre blanda': 8.294947121034077, 'Coliflor_Enferma_Pudrición del fruto': 14.308783783783785, 'Coliflor_Saludable_Saludable': 6.937592137592137, 'Fresa_Enferma_Hojas quemadas de la fresa': 1.2905329230019196, 'Fresa_Saludable_Saludable': 3.1361717882265827, 'Fríjol_Enferma_Mancha angular': 3.308389314169661, 'Fríjol_Enferma_Oxido de frijol': 3.279950437543561, 'Fríjol_Saludable_Saludable': 3.3470839260312943, 'Manzana_Enferma_Costra de la manzana': 2.2622583057365664, 'Manzana_Enferma_Hongo del manzano': 5.203194103194103, 'Manzana_Enferma_Pudrición del fruto': 2.3032247539289794, 'Manzana_Saludable_Saludable': 0.8698348804731784, 'Maíz_Enferma_Moho rojo del maíz': 1.1998980112187658, 'Maíz_Enferma_Pinta gris': 2.7919578114700068, 'Maíz_Enferma_Quemazón del maíz': 1.4526684044450542, 'Maíz_Saludable_Saludable': 1.2308631211857017, 'Papa_Enferma_Pudrición de la papa': 1.4308783783783783, 'Papa_Enferma_Quemado de la papa': 1.4308783783783783, 'Papa_Saludable_Saludable': 9.382809038546743, 'Pepino Cohombro_Enferma_Enferma': 4.088223938223938, 'Pepino Cohombro_Saludable_Saludable': 4.193050193050193, 'Tomate_Enferma_Hojas enrolladas del tomate': 0.26714182093411964, 'Tomate_Enferma_Hongo blanco del tomate': 1.5022345179825496, 'Tomate_Enferma_Moteado del tomate': 3.8412842372573914, 'Tomate_Enferma_Pinta en ojo de buey': 1.0193256479988448, 'Tomate_Enferma_Polvillo rojo': 0.8536187193905315, 'Tomate_Enferma_Pudrición de la papa': 0.7496415865767536, 'Tomate_Enferma_Puntitos marrones en las hojas': 0.8078353582940739, 'Tomate_Enferma_Puntos negros del tomate': 0.672958672958673, 'Tomate_Enferma_Quemado de la papa': 1.4308783783783783, 'Tomate_Saludable_Saludable': 0.8992165771427358, 'Yuca_Enferma_Manchas amarillas en la yuca': 0.10875001925733448, 'Yuca_Enferma_Manchas verdes en la yuca': 0.5996347316410177, 'Yuca_Enferma_Pudrición de la yuca': 1.3157502329916122, 'Yuca_Enferma_Venas marrones de la yuca': 0.6537422631083396, 'Yuca_Saludable_Saludable': 0.5554113065030095}
```

```
In [72]: datagen = ImageDataGenerator(
    validation_split = 0.2,
    preprocessing_function = None,
    rotation_range = 45,
    zoom_range = 0.2,
    horizontal_flip = True,
    vertical_flip = True,
    fill_mode = 'nearest',
    shear_range = 0.1,
    height_shift_range = 0.1,
    width_shift_range = 0.1
)

train_generator = datagen.flow_from_dataframe(
    dataframe=train,
    directory=data_dir,
    x_col='image_id',
```

```

y_col='Planta_Condición_Enfermedad',
target_size=(image_height, image_width),
batch_size=batch_size,
class_mode='categorical', # Importante para clasificación multiclase
subset='training',
seed=42,
classes=class_names # Asegurarse del orden de las clases
)

validation_generator = datagen.flow_from_dataframe(
    dataframe=train,
    directory=data_dir,
    x_col='image_id',
    y_col='Planta_Condición_Enfermedad',
    target_size=(image_height, image_width),
    batch_size=batch_size,
    class_mode='categorical',
    subset='validation',
    seed=42,
    classes=class_names
)

```

Found 33881 validated image filenames belonging to 37 classes.

Found 8470 validated image filenames belonging to 37 classes.

5. Cargar el modelo ResNet50 preentrenado (sin la capa superior de clasificación)

```
In [74]: base_model = ResNet50(weights='imagenet', include_top=False, input_shape=(in
```

6. Congelar las capas convolucionales base

```
In [76]: for layer in base_model.layers:
        layer.trainable = False
```

7. Añadir una nueva capa de clasificación personalizada

```
In [78]: x = base_model.output
x = GlobalAveragePooling2D()(x)
# Puedes añadir más capas densas aquí si lo deseas
# x = Dense(1024, activation='relu')(x)
predictions = Dense(num_classes, activation='softmax')(x)
```

8. Crear el modelo final

```
In [80]: model = Model(inputs=base_model.input, outputs=predictions)
```

9. Compilar el modelo para fine-tuning de la última capa

```
In [82]: model.compile(optimizer='adam',
                        loss='categorical_crossentropy',
                        metrics=['accuracy'])
```

10. Entrenar el modelo con la última capa descongelada

```
In [84]: epochs_fine_tune = 20 # Define el número de épocas para el fine-tuning
```

```
In [85]: print("Comenzando el entrenamiento de la última capa...")
history_fine_tune = model.fit(
    train_generator,
    steps_per_epoch=train_generator.samples // train_generator.batch_size,
    validation_data=validation_generator,
    validation_steps=validation_generator.samples // validation_generator.batch_size,
    epochs=epochs_fine_tune,
    verbose = 1
)
```

Comenzando el entrenamiento de la última capa...

Epoch 1/20

2258/2258 ————— 758s 335ms/step - accuracy: 0.6630 - loss: 1.1444 - val_accuracy: 0.7773 - val_loss: 0.7624

Epoch 2/20

2258/2258 ————— 154s 68ms/step - accuracy: 0.8000 - loss: 0.7862 - val_accuracy: 0.7762 - val_loss: 0.7760

Epoch 3/20

2258/2258 ————— 794s 351ms/step - accuracy: 0.7777 - loss: 0.6796 - val_accuracy: 0.7870 - val_loss: 0.6726

Epoch 4/20

2258/2258 ————— 157s 70ms/step - accuracy: 0.8000 - loss: 0.2877 - val_accuracy: 0.7892 - val_loss: 0.6639

Epoch 5/20

2258/2258 ————— 789s 349ms/step - accuracy: 0.7950 - loss: 0.6421 - val_accuracy: 0.7973 - val_loss: 0.6485

Epoch 6/20

2258/2258 ————— 157s 69ms/step - accuracy: 0.8667 - loss: 0.5161 - val_accuracy: 0.7957 - val_loss: 0.6438

Epoch 7/20
2258/2258 ————— **801s** 355ms/step – accuracy: 0.8117 – loss: 0.5893 – val_accuracy: 0.7770 – val_loss: 0.7429
 Epoch 8/20
2258/2258 ————— **162s** 72ms/step – accuracy: 0.6667 – loss: 1.0438 – val_accuracy: 0.7850 – val_loss: 0.7132
 Epoch 9/20
2258/2258 ————— **812s** 360ms/step – accuracy: 0.8153 – loss: 0.5771 – val_accuracy: 0.7947 – val_loss: 0.6422
 Epoch 10/20
2258/2258 ————— **166s** 73ms/step – accuracy: 0.8667 – loss: 0.5550 – val_accuracy: 0.7891 – val_loss: 0.6603
 Epoch 11/20
2258/2258 ————— **843s** 373ms/step – accuracy: 0.8175 – loss: 0.5678 – val_accuracy: 0.8080 – val_loss: 0.6150
 Epoch 12/20
2258/2258 ————— **164s** 72ms/step – accuracy: 0.7333 – loss: 1.2265 – val_accuracy: 0.8065 – val_loss: 0.6150
 Epoch 13/20
2258/2258 ————— **867s** 384ms/step – accuracy: 0.8198 – loss: 0.5520 – val_accuracy: 0.8080 – val_loss: 0.6611
 Epoch 14/20
2258/2258 ————— **166s** 73ms/step – accuracy: 0.8667 – loss: 0.4401 – val_accuracy: 0.8077 – val_loss: 0.6708
 Epoch 15/20
2258/2258 ————— **798s** 353ms/step – accuracy: 0.8182 – loss: 0.5712 – val_accuracy: 0.8148 – val_loss: 0.6081
 Epoch 16/20
2258/2258 ————— **153s** 68ms/step – accuracy: 0.8667 – loss: 0.2494 – val_accuracy: 0.8141 – val_loss: 0.6042
 Epoch 17/20
2258/2258 ————— **777s** 344ms/step – accuracy: 0.8225 – loss: 0.5541 – val_accuracy: 0.8115 – val_loss: 0.5925
 Epoch 18/20
2258/2258 ————— **153s** 68ms/step – accuracy: 0.8000 – loss: 0.5569 – val_accuracy: 0.8059 – val_loss: 0.6086
 Epoch 19/20
2258/2258 ————— **1933s** 856ms/step – accuracy: 0.8243 – loss: 0.5499 – val_accuracy: 0.7914 – val_loss: 0.7064
 Epoch 20/20
2258/2258 ————— **145s** 64ms/step – accuracy: 0.9333 – loss: 0.1722 – val_accuracy: 0.7820 – val_loss: 0.7393

```
In [86]: fin = datetime.datetime.now()
print(f"Fin: {fin.strftime('%Y-%m-%d %H:%M:%S.%f')}")

duracion = fin - inicio
print(f"Duración: {duracion}")
```

Fin: 2025-05-12 07:22:44.613336

Duración: 2:59:10.215743

11. Opcional: Descongelar algunas de las últimas capas del modelo base para un fine-tuning más profundo

```
In [88]: # unfreeze_layers = 5 # Número de capas a descongelar (contando desde la última)

# for layer in base_model.layers[-unfreeze_layers:]:
#     layer.trainable = True
```

12. Opcional: Recompilar el modelo después de descongelar las capas

```
In [90]: #model.compile(optimizer=tf.keras.optimizers.Adam(learning_rate=1e-5), # Usar Adam
#                    loss='categorical_crossentropy',
#                    metrics=['accuracy'])
```

13. Opcional: Entrenar el modelo con las capas descongeladas

```
In [92]: #epochs_deep_fine_tune = 5 # Define el número de épocas para el fine-tuning

#print(f"\nComenzando el fine-tuning profundo de las últimas {unfreeze_layers} capas")
#history_deep_fine_tune = model.fit(
#    train_generator,
#    steps_per_epoch=train_generator.samples // train_generator.batch_size,
#    validation_data=validation_generator,
#    validation_steps=validation_generator.samples // validation_generator.batch_size,
#    epochs=epochs_deep_fine_tune
#)
```

14. Evaluar el modelo final

```
In [94]: print("\nEvaluando el modelo...")
evaluation_result = model.evaluate(validation_generator)
print(f"Pérdida en la validación: {evaluation_result[0]:.4f}")
print(f"Precisión en la validación: {evaluation_result[1]:.4f}")
```

Evaluando el modelo...

565/565 ————— **145s** 256ms/step – accuracy: 0.7877 – loss: 0.7201

Pérdida en la validación: 0.7192

Precisión en la validación: 0.7857

```
In [95]: # Acceder a los datos de pérdida y precisión
loss = history_fine_tune.history['loss']
val_loss = history_fine_tune.history['val_loss']
accuracy = history_fine_tune.history['accuracy']
val_accuracy = history_fine_tune.history['val_accuracy']

# Graficar la pérdida del modelo
plt.figure(figsize=(12, 4))
plt.subplot(1, 2, 1)
plt.plot(history_fine_tune.history['loss'], label='Training Loss')
plt.plot(history_fine_tune.history['val_loss'], label='Validation Loss')
plt.title('Loss over epochs')
plt.xlabel('Epochs')
plt.ylabel('Loss')
plt.legend()

# Graficar la precisión del modelo
plt.subplot(1, 2, 2)
plt.plot(history_fine_tune.history['accuracy'], label='Training Accuracy')
plt.plot(history_fine_tune.history['val_accuracy'], label='Validation Accuracy')
plt.title('Accuracy over epochs')
plt.xlabel('Epochs')
plt.ylabel('Accuracy')
plt.legend()
```

Out[95]: <matplotlib.legend.Legend at 0x120ada6f0>



```
In [96]: epochs = list(range(1, 21))
data = {
    'Epoch': epochs,
```

```
'Training Loss': loss,  
'Validation Loss': val_loss,  
'Training Accuracy': accuracy,  
'Validation Accuracy': val_accuracy  
}  
df = pd.DataFrame(data)  
  
# Imprimir la tabla  
print(df)
```

	Epoch	Trainning Loss	Validation Loss	Training Accuracy \
0	1	0.872874	0.762390	0.725890
1	2	0.786222	0.775988	0.800000
2	3	0.668960	0.672626	0.783411
3	4	0.287728	0.663855	0.800000
4	5	0.635354	0.648465	0.796699
5	6	0.516077	0.643787	0.866667
6	7	0.602174	0.742914	0.809662
7	8	1.043779	0.713176	0.666667
8	9	0.589339	0.642159	0.811699
9	10	0.555001	0.660267	0.866667
10	11	0.581376	0.614981	0.815420
11	12	1.226480	0.614960	0.733333
12	13	0.570669	0.661108	0.817103
13	14	0.440109	0.670757	0.866667
14	15	0.565622	0.608062	0.821414
15	16	0.249372	0.604209	0.866667
16	17	0.564446	0.592475	0.821178
17	18	0.556942	0.608642	0.800000
18	19	0.560328	0.706382	0.823422
19	20	0.172151	0.739261	0.933333

	Validation Accuracy
0	0.777305
1	0.776241
2	0.786998
3	0.789244
4	0.797281
5	0.795745
6	0.776950
7	0.784988
8	0.794681
9	0.789125
10	0.808038
11	0.806501
12	0.808038
13	0.807683
14	0.814775
15	0.814066
16	0.811466
17	0.805910
18	0.791371
19	0.782033

```
In [97]: # Accuracy final en entrenamiento
val_loss = history_fine_tune.history['val_loss'][-1]
```

```
In [98]: print('Perdida final en Train:', history_fine_tune.history['loss'][-1])
print("Pérdida final en Validation:", history_fine_tune.history['val_loss'][-1])
```

```
print("Accuracy final en Train:", history_fine_tune.history['accuracy'][-1])
print("Accuracy final en Validation:", history_fine_tune.history['val_accu
```

Perdida final en Train: 0.17215093970298767
 Pérdida final en Validation: 0.7392613887786865
 Accuracy final en Train: 0.9333333373069763
 Accuracy final en Validation: 0.7820330858230591

```
In [99]: class_indices = train_generator.class_indices # o test_generator.class_indi
index_to_class = {v: k for k, v in class_indices.items()}
print(index_to_class)
with open("index_to_class_modelo14.json", "w") as f:
    json.dump(index_to_class, f, indent=4, ensure_ascii=False)
```

```
{0: 'Yuca_Enferma_Pudrición de la yuca', 1: 'Yuca_Enferma_Manchas amarillas
en la yuca', 2: 'Yuca_Enferma_Venas marrones de la yuca', 3: 'Yuca_Enferma_M
anchas verdes en la yuca', 4: 'Yuca_Saludable_Saludable', 5: 'Fresa_Saludabl
e_Saludable', 6: 'Papa_Enferma_Quemado de la papa', 7: 'Maíz_Saludable_Salud
able', 8: 'Tomate_Enferma_Pinta en ojo de buey', 9: 'Papa_Enferma_Pudrición
de la papa', 10: 'Tomate_Enferma_Pudrición de la papa', 11: 'Tomate_Enferma_
Moteado del tomate', 12: 'Tomate_Enferma_Hongo blanco del tomate', 13: 'Manz
ana_Enferma_Hongo del manzano', 14: 'Tomate_Enferma_Puntos negros del tomat
e', 15: 'Tomate_Enferma_Quemado de la papa', 16: 'Maíz_Enferma_Moho rojo del
maíz', 17: 'Tomate_Saludable_Saludable', 18: 'Tomate_Enferma_Hojas enrollada
s del tomate', 19: 'Manzana_Enferma_Costra de la manzana', 20: 'Maíz_Enferma
_Quemazón del maíz', 21: 'Tomate_Enferma_Polvillo rojo', 22: 'Tomate_Enferma
_Puntitos marrones en las hojas', 23: 'Maíz_Enferma_Pinta gris', 24: 'Manzan
a_Enferma_Pudrición del fruto', 25: 'Manzana_Saludable_Saludable', 26: 'Fres
a_Enferma_Hojas quemadas de la fresa', 27: 'Papa_Saludable_Saludable', 28: '
Coliflor_Enferma_Moho azu', 29: 'Pepino Cohombro_Saludable_Saludable', 30: '
Fríjol_Enferma_Mancha angular', 31: 'Coliflor_Saludable_Saludable', 32: 'Col
iflor_Enferma_Pudrición del fruto', 33: 'Fríjol_Enferma_Oxido de frijol', 3
4: 'Coliflor_Enferma_Podredumbre blanda', 35: 'Fríjol_Saludable_Saludable',
36: 'Pepino Cohombro_Enferma_Enferma'}
```

15. Testeo del Modelo

```
In [101]: test.Planta = test.Planta.astype('str')

test_generator = datagen.flow_from_dataframe(test,
    directory = "Datos Finales",
    subset = "training",
    x_col = "image_id",
    y_col = "Planta_Condición_Enfermedad",
    target_size = (image_height,image_width),
    batch_size = batch_size,
    class_mode = "categorical")
```

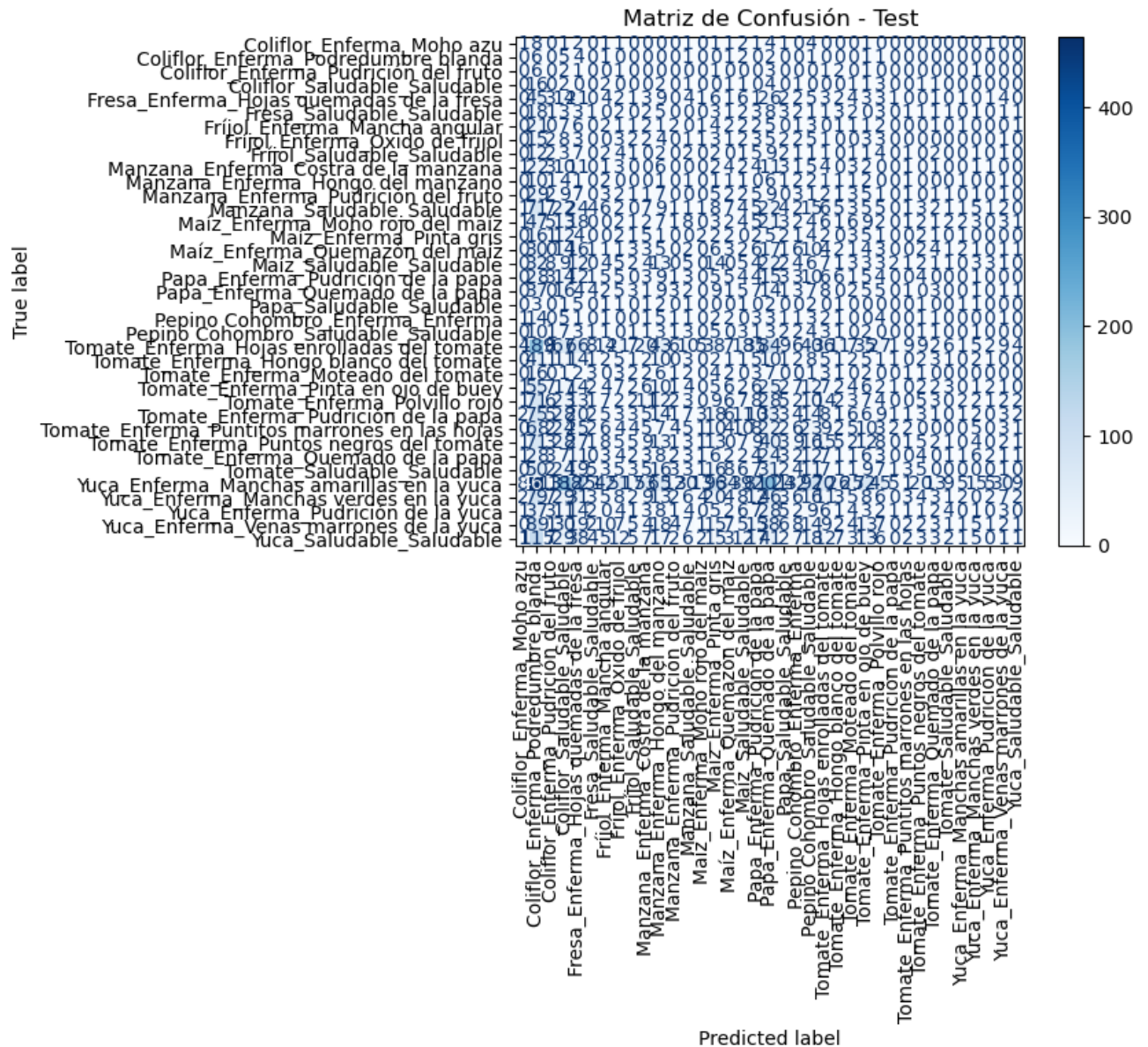
Found 8471 validated image filenames belonging to 37 classes.

```
In [102... # -----  
# 1. Obtener predicciones completas para test  
# -----  
y_pred_probs = model.predict(test_generator)  
y_pred = np.argmax(y_pred_probs, axis=1)  
y_true = test_generator.classes
```

565/565 ————— 147s 259ms/step

```
In [103... # -----  
# 2. Matriz de Confusión  
# -----  
plt.figure(figsize=(200, 80))  
cm = confusion_matrix(y_true, y_pred)  
disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=test_gener  
disp.plot(cmap="Blues", xticks_rotation=90)  
plt.title("Matriz de Confusión - Test")  
plt.savefig("matriz_confusion_test_modelo_11.png", dpi=300, bbox_inches='tig  
plt.show()
```

<Figure size 20000x8000 with 0 Axes>



```
In [104...] accuracy = accuracy_score(y_true, y_pred)
print(f"Test Accuracy: {accuracy:.4f}")
```

Test Accuracy: 0.0159

```
In [105...] # Función para normalizar imágenes solo para visualización
def normalize_for_display(img):
    img_min = img.min()
    img_max = img.max()
    if img_max > 1.0 or img_min < 0.0:
        img = (img - img_min) / (img_max - img_min + 1e-7)
    return img

# Obtener nombres de clases
class_indices = test_generator.class_indices
class_names = list(class_indices.keys())
```

```
# Reiniciar el generador para empezar desde el principio
test_generator.reset()
x_batch, y_true_batch = next(test_generator)

# Obtener predicciones del modelo
y_pred_probs_batch = model.predict(x_batch)
y_pred_batch = np.argmax(y_pred_probs_batch, axis=1)
y_true_batch = np.argmax(y_true_batch, axis=1)

# Número de imágenes a mostrar (máximo el tamaño del batch)
num_samples = x_batch.shape[0]

# Mostrar imágenes con etiquetas
plt.figure(figsize=(15, 8))
for i in range(num_samples):
    plt.subplot(3, 4, i + 1)
    img_to_show = normalize_for_display(x_batch[i])
    plt.imshow(img_to_show)
    plt.axis('off')
    plt.title(
        f"Real: {class_names[y_true_batch[i]]}\nPred: {class_names[y_pred_batch[i]]} "
        f"color='green' if y_true_batch[i] == y_pred_batch[i] else 'red' "
    )
plt.tight_layout()
plt.suptitle("Muestreo de Predicciones vs. Etiquetas Reales", fontsize=16, y=0.95)
plt.show()
```

1/1 ————— 1s 770ms/step

```

-----
ValueError                                Traceback (most recent call last)
Cell In[105], line 28
    26 plt.figure(figsize=(15, 8))
    27 for i in range(num_samples):
--> 28     plt.subplot(3, 4, i + 1)
    29     img_to_show = normalize_for_display(x_batch[i])
    30     plt.imshow(img_to_show)

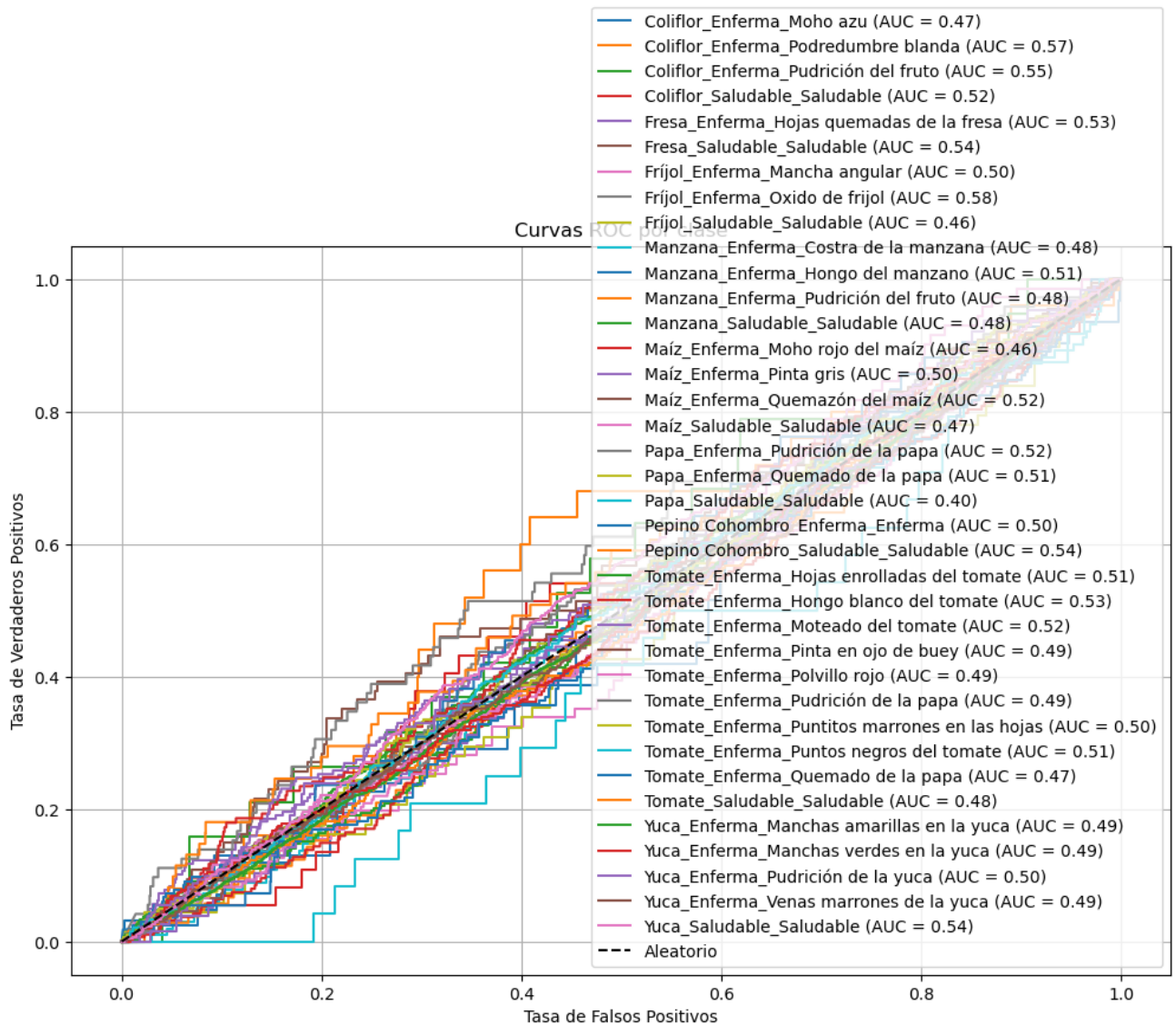
File /opt/anaconda3/lib/python3.12/site-packages/matplotlib/pyplot.py:1534,
in subplot(*args, **kwargs)
    1531 fig = gcf()
    1533 # First, search for an existing subplot with a matching spec.
-> 1534 key = SubplotSpec._from_subplot_args(fig, args)
    1536 for ax in fig.axes:
    1537     # If we found an Axes at the position, we can reuse it if the us
er passed no
    1538     # kwargs or if the Axes class and kwargs are identical.
    1539     if (ax.get_subplotspec() == key
    1540         and (kwargs == {}
    1541             or (ax._projection_init
    1542                 == fig._process_projection_requirements(**kwargs)))
    ):

File /opt/anaconda3/lib/python3.12/site-packages/matplotlib/gridspec.py:589,
in SubplotSpec._from_subplot_args(figure, args)
    587 else:
    588     if not isinstance(num, Integral) or num < 1 or num > rows*cols:
--> 589         raise ValueError(
    590             f"num must be an integer with 1 <= num <= {rows*cols}, "
    591             f"not {num!r}"
    592         )
    593     i = j = num
    594 return gs[i-1:j]

ValueError: num must be an integer with 1 <= num <= 12, not 13

```

Página 17 de 21



```
In [108... from sklearn.metrics import roc_auc_score

# AUC macro (promedio de clases, todas igual de importantes)
auc_macro = roc_auc_score(y_true_onehot, y_pred_probs, average='macro')

# AUC micro (promedia sobre todas las instancias)
auc_micro = roc_auc_score(y_true_onehot, y_pred_probs, average='micro')

print(f"AUC Macro promedio: {auc_macro:.4f}")
print(f"AUC Micro promedio: {auc_micro:.4f}")
```

AUC Macro promedio: 0.5021

AUC Micro promedio: 0.4932

```
In [110... # Asegúrate de tener esto:
# y_pred = np.argmax(y_pred_probs, axis=1)
# y_true = test_generator.classes
```

```

# Accuracy
accuracy = accuracy_score(y_true, y_pred)

# F1 Scores
f1_macro = f1_score(y_true, y_pred, average='macro')
f1_micro = f1_score(y_true, y_pred, average='micro')
f1_weighted = f1_score(y_true, y_pred, average='weighted')

print(f"Accuracy: {accuracy:.4f}")
print(f"F1 Score (macro): {f1_macro:.4f}")
print(f"F1 Score (micro): {f1_micro:.4f}")
print(f"F1 Score (weighted): {f1_weighted:.4f}")

# Detalle por clase
class_names = list(test_generator.class_indices.keys())
print("\nReporte de clasificación por clase:")
print(classification_report(y_true, y_pred, target_names=class_names))

```

Accuracy: 0.0159
 F1 Score (macro): 0.0131
 F1 Score (micro): 0.0159
 F1 Score (weighted): 0.0153

Reporte de clasificación por clase:

		precision	recall	f1-score
support				
31	Coliflor_Enferma_Moho azu	0.04	0.03	0.03
25	Coliflor_Enferma_Podredumbre blanda	0.00	0.24	0.01
19	Coliflor_Enferma_Pudrición del fruto	0.00	0.00	0.00
37	Coliflor_Saludable_Saludable	0.00	0.05	0.01
179	Fresa_Enferma_Hojas quemadas de la fresa	0.03	0.12	0.05
74	Fresa_Saludable_Saludable	0.01	0.01	0.01
71	Fríjol_Enferma_Mancha angular	0.01	0.03	0.02
72	Fríjol_Enferma_Oxido de frijol	0.02	0.04	0.02
68	Fríjol_Saludable_Saludable	0.01	0.01	0.01
102	Manzana_Enferma_Costra de la manzana	0.00	0.00	0.00
42	Manzana_Enferma_Hongo del manzano	0.00	0.02	0.00
	Manzana_Enferma_Pudrición del fruto	0.00	0.00	0.00

105	Manzana_Saludable_Saludable	0.01	0.00	0.01
263	Maíz_Enferma_Moho rojo del maíz	0.00	0.00	0.00
188	Maíz_Enferma_Pinta gris	0.01	0.03	0.01
77	Maíz_Enferma_Quemazón del maíz	0.02	0.02	0.02
153	Maíz_Saludable_Saludable	0.03	0.03	0.03
182	Papa_Enferma_Pudrición de la papa	0.01	0.03	0.02
160	Papa_Enferma_Quemado de la papa	0.02	0.09	0.03
160	Papa_Saludable_Saludable	0.00	0.00	0.00
24	Pepino Cohombro_Enferma_Enferma	0.01	0.02	0.01
55	Pepino Cohombro_Saludable_Saludable	0.01	0.07	0.02
61	Tomate_Enferma_Hojas enrolladas del tomate	0.13	0.04	0.06
869	Tomate_Enferma_Hongo blanco del tomate	0.01	0.01	0.01
145	Tomate_Enferma_Moteado del tomate	0.00	0.00	0.00
63	Tomate_Enferma_Pinta en ojo de buey	0.02	0.03	0.02
228	Tomate_Enferma_Polvillo rojo	0.02	0.01	0.02
276	Tomate_Enferma_Pudrición de la papa	0.06	0.00	0.01
316	Tomate_Enferma_Puntitos marrones en las hojas	0.06	0.01	0.01
284	Tomate_Enferma_Puntos negros del tomate	0.06	0.02	0.02
331	Tomate_Enferma_Quemado de la papa	0.00	0.00	0.00
165	Tomate_Saludable_Saludable	0.00	0.00	0.00
259	Yuca_Enferma_Manchas amarillas en la yuca	0.21	0.00	0.00
2051	Yuca_Enferma_Manchas verdes en la yuca	0.00	0.00	0.00
372	Yuca_Enferma_Pudrición de la yuca	0.00	0.00	0.00
182	Yuca_Enferma_Venas marrones de la yuca	0.02	0.01	0.01
358	Yuca_Saludable_Saludable	0.03	0.00	0.00

424

	accuracy			0.02
8471				
	macro avg	0.02	0.03	0.01
8471				
	weighted avg	0.08	0.02	0.02
8471				

16. Guardar el modelo entrenado

```
In [113... model.save('modelo_14_resnet50_fine_tuned_class_weighted.h5')
```

WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save\_model(model)`. This file format is considered legacy. We recommend using instead the native Keras format, e.g. `model.save('my\_model.keras')` or `keras.saving.save\_model(model, 'my\_model.keras')`.

```
In [115... class_indices = train_generator.class_indices # o test_generator.class_indices
index_to_class = {v: k for k, v in class_indices.items()}
```

```
In [ ]:
```